

FITLY Wireframe 및 UX 설계 전략 브리핑

왜 이 화면 구조인가 · Customer Single-page · Admin 운영 효율 · Enterprise Human-in-the-loop · 구현/운영 비용 · 확장성

2026.09.03
FITLY UX / Frontend 설계

설계 한 줄 요약 · FITLY는 화면 수를 늘리는 대신 Actor별 핵심 업무를 짧은 흐름으로 묶는다. Customer는 고정 Header/Navigation 아래 Main Content만 전환하는 Single-page 경험으로 추천 진입 장벽을 낮추고, Admin은 많은 상태를 비교해야 하므로 Table 중심으로, Enterprise는 상품 사진 → AI 초안 → MD 수 정 → 등록을 한 화면에서 끝내도록 설계한다. 핵심 원칙은 “사용자의 다음 행동이 항상 보이고, AI 결과는 확인·수정 가능하며, 화면 구조가 API/DB 구조와 자연스럽게 연결되는 것”이다.



1. 왜 이런 Wireframe 구조를 선택했는가?

FITLY의 핵심은 “많은 메뉴를 탐색하는 쇼핑물”이 아니라 **상황을 입력하고, 보유 옷을 활용한 코디를 빠르게 결정한 뒤 필요한 상품만 대여하는 것**이다. 따라서 페이지 수보다 사용자의 의사결정 흐름을 기준으로 화면을 설계했다.

Customer: AI 추천이 Primary CTA. Dashboard보다 추천 진입을 첫 화면의 중심에 둔다.

Single-page: 공통 Header/Navigation을 유지하고 Main Content Component만 바꿔 맥락 손실과 페이지 이동을 줄인다.

Actor 분리: Customer-Admin-Enterprise는 목적과 정보 밀도가 달라 동일 UI 패턴을 강제로 공유하지 않는다.

MVP 원칙: 독립 페이지·Actor를 과도하게 늘리지 않고 실제 운영에 필요한 흐름만 남긴다.

핵심 판단 · 화면을 기능별로 잘게 쪼개는 대신, “한 업무를 완료하는 데 필요한 화면 수”를 최소화했다.

2. Customer 입력 화면의 결정은 왜 타당한가?

결정	채택 구조	이유
TPO / Style	Chip 선택	서로 비교하는 소수 선택지는 Dropdown보다 즉시성이 높고 현재 상태를 한눈에 확인 가능.
내 옷 입력	사진 업로드 + 내 옷장 불러오기	신규 사용자와 기존 옷장 사용자 모두 같은 추천 Flow에 진입 가능.
AI 인식값	Thumbnail + 결과 확인·수정	Vision 오류를 사용자가 즉시 보정하는 Human-in-the-loop. 잘못된 인식이 추천 전체로 전파되는 것을 차단.
Size / 요약	우측 고정 입력 + 선택 요약	추천 요청 직전 조건 누락·오입력을 확인하고 필수 입력 완료 시에만 CTA 활성화.

3. 추천 결과 화면은 왜 “내 옷 vs 대어 상품”으로 나누는가?

FITLY의 차별점은 새로운 옷을 많이 추천하는 것이 아니라 이미 가진 옷을 0원 자산으로 활용하고 부족한 부분만 대여하는 것이다. 따라서 추천 카드에서 소유 자산과 대여 비용을 시각적으로 분리해야 서비스 가치가 바로 전달된다.

4:5 세로형 코디 카드로 상의~하의 전체 조합을 먼저 비교.

Match / TPO / Style / 필요 대어 금액을 카드 단계에서 제공해 상세 진입 전 의사결정 지원.

저장(♡)과 Filter는 현재 화면의 State 변화로 처리하고, 상세보기만 다음 Component로 전환.

상세에서는 상품 판단과 대어 CTA를 함께 배치해 결제 전 불필요한 페이지 증가를 피함.

4. Admin과 Enterprise는 왜 다른 UI 패턴을 사용하는가?

Actor	UI 원칙	설계 이유
Admin	Dashboard 요약 + Table	주문·상품·반납 상태를 동시에 많이 비교하므로 Card보다 검색/필터가 있는 Table이 운영 효율적.
Return	반납→검수→세탁→재고복구	현재는 하나의 운영 연속 과정이므로 별도 Actor/페이지로 과분리하지 않고 상태 Flow로 관리.
Enterprise	한 Page Form	사진, 기본 정보, AI 설명, 수정, 등록을 같은 맥락 안에서 완료해 상품 등록 시간을 줄임.
AI Description	Textarea 편집 가능	AI는 Draft만 생성하고 최종 책임은 MD가 갖도록 하여 과장·오인 정보를 통제.

핵심 원칙 · Customer는 선택을 쉽게, Admin은 비교를 빠르게, Enterprise는 등록을 짧게 만든다.

5. 구현·운영 관점에서 Wireframe을 어떻게 단순화했는가?

Component 전환 Header/GNB를 유지하고 Main 영역만 교체. 라우팅·상태 복 잡도와 중복 레이아웃을 줄인다.	Structured Binding AI 결과를 자유문장이 아닌 정형 필드로 받아 LOOK 카드, 내 옷, 대어 상품, 가격에 직접 바인딩한다.	State 명문화 loading / empty / failed / saved / selected 같은 UI 상태를 Component 내부에서 명시해 운영 오류를 줄인다.
---	---	--

UI가 DB의 모든 컬럼을 노출하지 않고, 각 Actor가 행동에 필요한 정보만 보여준다. 재고·가격·상태는 화면의 임의 값이 아니라 Backend/API의 검증된 값을 표시하며, AI 인식·설명은 사용자가 수정 가능한 상태로 구분한다.

운영 기준 · “화면에 보이는 상태”와 “서버의 실제 상태”가 분리되지 않도록 CTA를 API 상태와 연결한다.

6. 비용·배송·확장을 고려하면 어떤 효과가 있는가?

개발 비용 공통 Layout과 Component 상태 전환을 재사용해 페이지·라우트·중복 코드 증가를 억제한다.	운영 비용 초기부터 복잡한 Analytics/Actor/실시간 UI를 추가하지 않고 주문·재고·반납의 핵심 예외만 노출한다.	배송/확장 Vue SPA + Spring Boot API 구조를 유지하고, 트래픽이나 업무가 커질 때 필요한 Component/도메인만 단계적으로 분리한다.
--	---	---

Wireframe 단계에서 화면을 과도하게 확장하지 않는 것은 단순한 디자인 취향이 아니라, API 수·상태 관리·테스트 케이스·배송 리스크까지 줄이는 엔지니어링 선택이다.

결론 · FITLY Wireframe은 “많은 화면을 그린 설계”가 아니라 Customer의 추천·대여, Admin의 운영, Enterprise의 등록이라는 서로 다른 업무를 가장 짧은 의사결정 흐름으로 연결한 설계다. Single-page 전환 + Actor별 UI 패턴 + Human-in-the-loop + Structured API Binding으로 사용성, 개발 생산성, 운영 안정성을 동시에 확보한다.